

Project 3 Antlr Design Report

CS-4240 – Compilers & Interpreters

David Folger, Devin Fromond, Rupesh Seetharaman

21 November, 2025

1. Implementation Strategy

Our group began by making the grammar clear and LL(1) by splitting the expressions into simple layers (or/and/compare/add/multiply/primary) so each operator happens in order. Additionally, we used right-recursive forms so groupings such as `a - b - c` form right-associatively – as per the project instructions. Anywhere an `ID` starts something, we factor the next token to decide: `ID(` refers to a call, `ID{` or `ID :=` refers to an assignment, and `ID` in the context of an expression refers to a variable. We ensure lists and optional are written as repeats or simply `?` so there is no left-recursion. Finally, the lexer follows the language tokens – keywords, operators, punctuation, numbers, IDs – while skipping comments/whitespace, and it matches `FLOAT` before `INT` so `12.3` is maintained as a single token.

2. Language and Build Instructions

Our project is in Java. We constructed `run.sh` so it is the only needed utility: it installs `antlr-4.13.2-complete.jar` if necessary, clears `antlr_generated` and `build`, regenerates lexer/parser from `tiger.g4`, compiles the required files, and runs `Main <file>`. In the event manual regeneration is required, run `./run.sh some.tiger` and it will rebuild from the grammar.

3. Challenges and Solutions

Right-associative operators are difficult to create and interpret for LL(1) if they are written directly. We find the layered, right-recursive rules keep the correct grouping and ensure no conflicts. Additionally, `ID` was ambiguous (call vs assign vs array element), so the decision to split the *tail* after `ID` based on the next symbol was made to maintain determinism. To resolve errors with limited troubleshooting overhead provided, we swapped ANTLR's default listeners to a minimal one to print `[LexicalError]` and `[SyntaticError]` prefixes while letting ANTLR recover, ensuring multiple errors in a single pass are visible. Finally, through trial and error, we discovered that token order matters; putting `FLOAT` before `INT` and keywords before `ID` resolves errors related to mis-lexing.

4. Bugs and Deficiencies

All provided `test_cases` print `successful`. In addition, provided lexical/syntactic error cases show clear tagged messages. We have no known missing language constructs. We have included additional private test cases for additional verification in `new_test_cases`.